

EXPERIMENT – 3: MULTIPLE LINEAR REGRESSION (Using California Housing Dataset)

1. Aim

To develop a Multiple Linear Regression model for the given dataset, evaluate its performance and predict continuous values.

2. Objective

- Understand multiple regression concept
- Predict continuous values using multiple input features
- Train regression model using dataset
- Evaluate model performance using MSE and R^2 score
- Interpret regression coefficients

3. Theory

What is Regression?

Regression is a supervised learning technique used to predict continuous values.

Examples

- House price prediction
- Salary prediction
- Temperature prediction

What is Multiple Linear Regression?

Multiple Linear Regression predicts output using multiple input variables.

Mathematical Equation

$$y=b_0+b_1x_1+b_2x_2+b_3x_3+...+b_nx_n$$

Where:

- $y \rightarrow$ predicted value
- $b_0 \rightarrow$ intercept
- $b_1, b_2 \dots b_n \rightarrow$ coefficients
- $x_1, x_2 \dots x_n \rightarrow$ input features

Advantages

- ✓ Uses multiple features for prediction
- ✓ Better accuracy than simple regression
- ✓ Shows relationship between variables

4. Algorithm

1. Load dataset
2. Separate input and output variables
3. Split data into training and testing sets
4. Train multiple linear regression model
5. Predict test data
6. Evaluate performance using MSE and R^2
7. Display coefficients

5. Procedure

- Import required libraries
- Load California housing dataset
- Perform train-test split
- Train linear regression model
- Predict output values
- Evaluate performance

6. Complete Python Code

```
[1]: # Import required libraries
import pandas as pd
import numpy as np

# Load dataset
from sklearn.datasets import fetch_california_housing

# Train test split
from sklearn.model_selection import train_test_split

# Multiple Linear Regression model
from sklearn.linear_model import LinearRegression

# Evaluation metrics
from sklearn.metrics import mean_squared_error, r2_score
```

```
[2]: # Step 1: Load Dataset
data = fetch_california_housing(as_frame=True)
df = data.frame

print("Dataset Preview:")
print(df.head())
```

Dataset Preview:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	MedHouseVal
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

```
[3]: # Step 2: Separate input and output
X = df.drop("MedHouseVal", axis=1)
y = df["MedHouseVal"]

[4]: # Step 3: Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

[5]: # Step 4: Train Model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
[5]: LinearRegression ⓘ ?
LinearRegression()
```

```
[6]: # Step 5: Prediction
y_pred = model.predict(X_test)

[7]: # Step 6: Evaluate Performance
print("\nModel Performance")
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

Model Performance
MSE: 0.555891598695244
R2 Score: 0.5757877060324511

[8]: # Step 7: Display Coefficients
print("\nModel Coefficients:")
print(model.coef_)

Model Coefficients:
[ 4.48674910e-01  9.72425752e-03 -1.23323343e-01  7.83144907e-01
 -2.02962058e-06 -3.52631849e-03 -4.19792487e-01 -4.33708065e-01]
```

7. Output Interpretation

Mean Squared Error (MSE)

- Lower value → better model performance.

R² Score

- Range: 0 to 1
- Closer to 1 → good prediction.

Coefficients

- Show importance of each feature in prediction.